A PROJECT REPORT ON

Stock Price and Nifty 50 Index Forecasting Using LNN and BiLSTM based Deep Learning Models

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Anushka Adsure	**Exam No: B400050316**
Atharv Despande	**Exam No: B400050326**
Geetali Bendale	**Exam No: B400050336**
Prathamesh Bhawar	**Exam No: B400050340**

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

Stock Price and Nifty 50 Index Forecasting Using LNN and BiLSTM based Deep Learning Models

Submitted by

Anushka Adsure	Exam No: B400050316
Atharv Despande	Exam No: B400050326
Geetali Bendale	Exam No: B400050336
Prathamesh Bhawar	Exam No: B400050340

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. R.A.Kulkarni** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. R.A.Kulkarni
Internal Guide
Dept. of Computer Engg.

Dr. G.V.Kale
Head
Dept. of Computer Engg.

Dr. S.T.Gande
Principal
Pune Institute of Computer Technology

Place:Pune
Date:02/05/2025

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'Stock Price and Nifty 50 Index Forecasting Using LNN and BiLSTM based Deep Learning Models'****.*

We would like to take this opportunity to thank ***Dr. R.A.Kulkarni*** *giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

We are also grateful to ***Dr. G. V. Kale****, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

We would like to express our heartfelt gratitude to ***Mr. Akshay Kunkulol*** *and the entire* ***Infinity Pool*** *team for their continuous support, insightful guidance, and encouragement throughout the completion of this project. We also thank* ***Mr. Mahadev Valekar*** *for his valuable contributions and constant support during this journey.*

Anushka Adsure
Atharv Despande
Geetali Bendale
Prathamesh Bhawar
(B.E. Computer Engg.)

ABSTRACT

The Indian stock market is a complex, nonlinear system shaped by long-term trends, short-term volatility, and public sentiment. In this research, we present a dual-model forecasting system designed to predict short-term movements of individual NSE-listed company stocks and the Nifty index. Our approach utilizes over two decades of historical stock market data, applying a Liquid Neural Network (LNN) architecture for company-level stock prediction, and a Bidirectional LSTM (BiLSTM) model for forecasting the Nifty index. To enhance the predictive capacity of the Nifty model, we integrate sentiment analysis derived from real-time financial news sources, allowing the system to factor in market mood alongside technical price patterns. This framework addresses the core challenge of anticipating future price action in a highly volatile and sentiment-sensitive environment. By combining deep learning techniques with long-term historical data and market sentiment, the system captures both macroeconomic influences and micro-level stock behavior. The results show promising accuracy in both stock- and index-level forecasts, demonstrating the potential of hybrid time-series and sentiment-aware models in real-world financial prediction tasks.

Contents

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Overview

The stock market is inherently volatile and influenced by a range of factors including economic indicators, global trends, and investor sentiment. Forecasting future price movements, especially in the Indian stock market, requires a model capable of understanding both historical behavior and sudden fluctuations. To tackle this, our project introduces a two-model forecasting system designed to predict both individual stock prices and the broader Nifty 50 index using deep learning. For stock-level predictions, we employ a Liquid Neural Network (LNN) trained on over 20 years of historical data. To enhance the model's ability to learn patterns, we integrate a suite of technical indicators that capture critical aspects of price behavior such as momentum, volatility, volume strength, and trend reversals. These indicators help the LNN better interpret the internal structure of market cycles and refine its understanding of price dynamics, which purely raw price data might fail to convey. On the other hand, forecasting the Nifty index involves a Bidirectional Long Short-Term Memory (BiLSTM) network that combines historical price data with sentiment analysis derived from financial news. This hybrid approach allows the model to leverage not just past price movements, but also current market psychology, enhancing its prediction accuracy. Together, these models form a comprehensive solution for informed forecasting at both individual and market-wide levels.

1.2 Motivation

The motivation behind this project arises from the pressing need for reliable tools that can assist retail and institutional investors in making informed decisions in the Indian stock market. Traditional methods often fail to accommodate the market's dynamic nature, while modern models either overlook historical depth or underutilize the impact of public sentiment. By integrating deep learning models with long-term historical data and real-time sentiment analysis, we aimed to bridge this gap and provide a holistic, AI-powered forecasting framework capable of navigating the complexities of market prediction with greater accuracy and context-awareness.

1.3 Problem Definition

The project aims to develop deep learning models using Python to predict stock price values for the next trading week and to forecast the next-day

price value of the Nifty 50 index.

1.4 Objectives

The primary objectives of this project are:

- Predict the stock price values for the upcoming trading week using a deep learning model built in Python.
- Incorporate custom-engineered features into the model to improve flexibility and performance.
- Forecast the next-day price value of key market indices—Nifty 50, Bank Nifty, Fin Nifty, and Midcap Nifty.
- Integrate technical indicators and sentiment analysis to enhance prediction accuracy.
- Support more informed and data-driven investment decisions through reliable forecasting.

1.5 Project Scope & Limitations

1. This project focuses on developing a deep learning-based forecasting system to predict stock prices for the upcoming trading week and to forecast next-day price movements of key Indian indices such as Nifty 50. The model is built using Python, allowing the flexibility to integrate custom technical indicators and historical market sentiment. The system processes over 20 years of historical OHLCV data to learn temporal and statistical patterns, thereby supporting short-term financial decision-making. The project supports visualizing predicted trends and provides an extensible framework that can be applied to other stocks or indices with minimal modification.
2. While the model leverages historical data and technical indicators for forecasting, its performance can still be affected by unforeseen macroeconomic events, policy changes, or sudden market sentiment shifts not reflected in historical data. The predictions are limited to short-term horizons and assume the continuation of existing market behavior. The model also does not incorporate live news feeds or real-time market events during inference, which may influence actual price movements.

Additionally, although sentiment analysis is included for index prediction, its accuracy is dependent on the quality and coverage of the sentiment data used.

1.6 Methodologies of Problem solving

1. Problem Definition and Understanding
    - Goal: Clearly define the problem, understand the requirements, constraints, and desired outcomes.
    - Example: "Predict the next 7 days of Nifty index closing prices accurately using past market data."
2. Data Collection and Preprocessing
    - Goal: Gather relevant, high-quality data and clean it for analysis.
    - Steps: Handle missing values, remove outliers, normalize or scale features, extract useful indicators or features.
    - Example: Use OHLC data, volume, and PCA-based financial indicators. Add technical indicators if needed.
3. Model Selection and Design
    - Goal: Choose or design the right model architecture suited for the problem.
    - Steps: Try various models (e.g., LSTM, BiLSTM) and select based on task complexity and data characteristics.
    - Example: Use LSTM for sequence modeling, possibly extend to Seq2Seq for multi-day forecasts.
4. Training, Validation, and Evaluation
    - Goal: Train the model with appropriate hyperparameters and evaluate it using performance metrics.
    - Steps: Split data into training/validation, monitor loss and metrics like MAE, RMSE, R^2. Avoid overfitting.
    - Example: Train on historical Nifty data and evaluate performance on a held-out period.
5. Iteration and Optimization

- Goal: Continuously refine the solution based on feedback, results, and edge cases.
- Steps: Analyze where the model fails, tune hyperparameters, enrich features, or change modeling strategies.
- Example: If predictions deviate at the end, try longer input sequences, re-train on recent data, or adjust forecast strategy.

CHAPTER 2
LITERATURE SURVEY

Stock market prediction has long been a challenging yet fascinating problem due to its inherently noisy, dynamic, and non-linear behavior. Traditional forecasting models such as ARIMA are widely used in time series analysis but often fall short when applied to financial markets, where patterns can be influenced by both short-term fluctuations and long-term dependencies. These limitations have driven a shift toward more advanced machine learning and deep learning approaches for modeling stock price behavior.

In recent years, Layered Neural Networks (LNNs), also referred to as feed-forward deep neural networks, have gained popularity for their simplicity and effectiveness in capturing complex non-linear relationships between input features. In stock market prediction tasks, LNNs have been applied to historical OHLC (Open, High, Low, Close) data enriched with technical indicators like RSI, MACD, and ATR. The integration of these indicators allows the network to capture trend, momentum, and volatility aspects of market behavior. When trained with well-engineered features, LNNs can serve as efficient and fast predictors for short-term price movements of individual stocks.

On the other hand, forecasting broader market indices like the Nifty 50 requires capturing sequential dependencies and temporal dynamics over time. Bidirectional Long Short-Term Memory (BiLSTM) networks are well-suited for such tasks, as they can process time series data in both forward and backward directions, allowing the model to learn from past and future contexts simultaneously. This enhances the model's ability to detect patterns and anomalies in the historical Nifty index data. Studies have shown that BiLSTMs outperform traditional LSTMs and other time-series models in financial forecasting due to their contextual awareness.

To improve generalization and reduce the impact of noisy financial data, Principal Component Analysis (PCA) is often used in conjunction with these models. PCA helps in dimensionality reduction by extracting the most significant features from a high-dimensional feature space, especially when combining company-wise indicators. This not only reduces computation time but also ensures that the models focus on the most informative aspects of the data.

In summary, the combination of LNNs for individual stock prediction and BiLSTM networks for index-level forecasting leverages the strengths of both feedforward and recurrent architectures. When enriched with technical indicators and PCA-compressed features, this hybrid approach balances interpretability, accuracy, and computational efficiency. Continuous retraining

and feature refinement remain crucial to adapt to changing market conditions and ensure sustained model performance.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

1. Assumptions

   - Clean Historical Data Available
     It is assumed that the historical OHLC data for stocks and the Nifty index is already cleaned, formatted, and available for processing.
   - Static PCA Features
     For Nifty prediction, it is assumed that the PCA-compressed features are precomputed and do not require real-time updates.
   - User Understanding
     It is assumed that users have basic knowledge of stock prediction and can interpret forecast graphs and signals.
   - Model Suitability
     The BiLSTM model is assumed to be more effective for time-series data like the Nifty index, while LNN is assumed to be better for individual stock price forecasting using OHLC data.

2. Dependencies

   - Libraries and Frameworks
     The system depends on external Python libraries such as:
     - TensorFlow/Keras
     - NumPy
     - Pandas
     - Matplotlib
     - Scikit-learn

     These are used for model building, training, and visualization.
   - Frontend-Backend Integration
     The UI is dependent on seamless communication between the frontend (e.g., React) and the Python backend (Flask).
   - CSV Data Files
     The application relies on local or server-side CSV files for loading OHLC and PCA data for both training and inference.
   - Hardware/Runtime Environment
     The system depends on sufficient computational resources (CPU/GPU and RAM) to train or run prediction models smoothly without lag.

3.2 Functional Requirements

The following are the functional requirements for the proposed system:

3.2.1 System Feature 1: Forecasting Engine (Functional Requirement)

- **Description:** The system provides two forecasting options:
  a. Forecast Nifty index prices using BiLSTM model
  b. Forecast individual stock prices using OHLC data and technical indicators via a Layered Neural Network (LNN).
- **Functionalities:**
  - User selects between Nifty Index or Stock Forecasting.
  - For Nifty index, the model uses averaged PCA features and historical index data to predict the next 7 days.
  - For stock prediction, the model uses the last 60 days of OHLC + technical indicators to predict the closing price 5 days into the future.
  - Forecasts are displayed with actual date labels for clarity.
- **Input:** Company name or choice of "Nifty"
- **Output:** Predicted closing prices with corresponding future dates

3.2.2 System Feature 2: User Interface and Visualization (Functional Requirement)

- **Description:** The system provides a user-friendly UI with intuitive controls for company selection, forecast initiation, and result interpretation.
- **Functionalities:**
  - Clean interface with dropdown or input field to choose between Nifty or stock forecasting.
  - Graphical representation of predicted vs actual prices.
  - Confidence bands (optional) around predictions to visually represent model uncertainty.

  - Option to download prediction results in CSV format for future analysis.

- **Input:** User interaction through UI
- **Output:** Visual forecast (line graph), downloadable CSV file

3.3 External Interface Requirements

The following are the external interface requirements for the proposed system:

3.3.1 User Interfaces

- The system will provide a clean, responsive, and intuitive web-based user interface.
- Users will be able to:
  - Select between ”Nifty Forecasting” and ”Stock Forecasting”.
  - Input a stock name (or select from a dropdown).
  - View forecasted results on a graph (Actual vs Predicted).
  - Download the forecast results in CSV format.
- Forecasting output will include:
  - Predicted values with date mapping.
  - Optional confidence bands (for Nifty).
  - Tabs or toggles to switch between prediction modes.

3.3.2 Hardware Interfaces

- No specialized hardware is required.
- The system can run on:
  - Standard PCs or laptops with internet access.
  - Server environments for hosted versions (optional GPU for faster training).
- Recommended minimum:
  - 4 GB RAM, Dual-core CPU, 5 GB storage.

3.3.3 Software Interfaces

- **Operating System:** Compatible with Windows, Linux, or macOS.
- **Backend Environment:** Python 3.x, Flask/Django (for server-side logic).
- **Libraries:** TensorFlow/Keras, Pandas, NumPy, Scikit-learn, Matplotlib.
- **Frontend Stack:** HTML/CSS/JavaScript, optionally React for dynamic UI.
- **Browser Support:** Chrome, Firefox, Edge (latest versions).

3.3.4 Communication Interfaces

- **Local Deployment**
  - HTTP-based interaction between frontend and backend.
  - Localhost or server IP used for hosting Flask/Django app.
- **Data Input/Output**
  - CSV uploads and downloads through the browser interface.
  - No external APIs are currently used (can be added for real-time stock data).

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- The system shall provide predictions within 3–5 seconds of user input under normal load conditions.
- The UI shall respond to user interactions (e.g., dropdown selection, button clicks) within 1 second.
- The application shall be able to handle simultaneous access by multiple users without significant delays (for web deployment).
- Forecast models shall efficiently process datasets with over 5 years of historical data with consistent accuracy.

3.4.2 Safety Requirements

The system does not directly affect physical safety but ensures data integrity during processing and storage.

The application must avoid system crashes or data loss, especially during file upload or prediction execution.

Graceful error handling must be implemented to avoid unintentional shutdowns or user confusion.

3.4.3 Security Requirements

- The system must validate all inputs to prevent injection attacks or malformed data uploads.
- File uploads must be restricted to CSV format and scanned to prevent malicious execution.
- For hosted deployments:
  - HTTPS shall be enforced to secure data transmission.
  - Basic authentication and role-based access control can be implemented if user accounts are supported.
  - No sensitive user data is collected or stored by default.

3.4.4 Software Quality Attributes

Quality Attributes

- **Usability**: Simple and minimal UI for ease of use by users with non-technical backgrounds.
- **Maintainability**: Modular codebase, easy to update models or UI components independently.
- **Portability**: The application shall run on multiple OS platforms and can be easily migrated to a cloud server.
- **Reliability**: The application must consistently return accurate predictions based on valid historical data.
- **Scalability**: The backend can be upgraded to support larger datasets or more complex models without redesigning the system.

3.5 System Requirements

3.5.1 Database Requirements

The system does not rely on a traditional database like MySQL or MongoDB. Instead, it uses structured CSV files to manage historical stock and index data. These include files like `cleaned_nifty_data.csv` and various OHLC/PCA feature files for individual companies. Optionally, a lightweight database like SQLite or Firebase can be introduced for logging prediction history, managing user preferences, or future scalability.

3.5.2 Software Requirements (Platform Choice)

The backend of the system is developed in Python 3.12.6 or higher. Libraries and frameworks used include Pandas, NumPy, Matplotlib, TensorFlow/Keras for model implementation, and Scikit-learn for data preprocessing. Flask or Django is used for the web application backend, while the frontend may optionally use simple HTML/CSS or ReactJS. Development is typically done in environments like Jupyter Notebook, VS Code, or PyCharm. For deployment, platforms such as Heroku, PythonAnywhere, or AWS EC2 can be used.

3.5.3 Hardware Requirements

The minimum hardware required for running the application includes a dual-core CPU, 4 GB of RAM, and at least 5 GB of free disk space. However, for smoother execution and training of models, it is recommended to use a quad-core processor, 8 GB or more of RAM, and an SSD with at least 10 GB of free space. While a GPU is not mandatory, having an NVIDIA GPU significantly speeds up model training, especially for deep learning tasks.

3.6 Analysis Models: SDLC Model to be applied

The project follows the **Iterative Incremental Model** of the Software Development Life Cycle (SDLC). **Justification:**

1. **Modular Approach:** Stock and Nifty forecasting modules are developed and tested independently.
2. **Frequent Feedback:** Iterations allow continuous improvement based on visual output and prediction accuracy.
3. **Flexibility:** Easy to add new features like sentiment analysis, downloadable reports, or confidence bands.
4. **Parallel Development:** UI and prediction models can be built and updated separately.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 illustrates the architecture of the Liquid Neural Network (LNN) used for time-series prediction in this project.

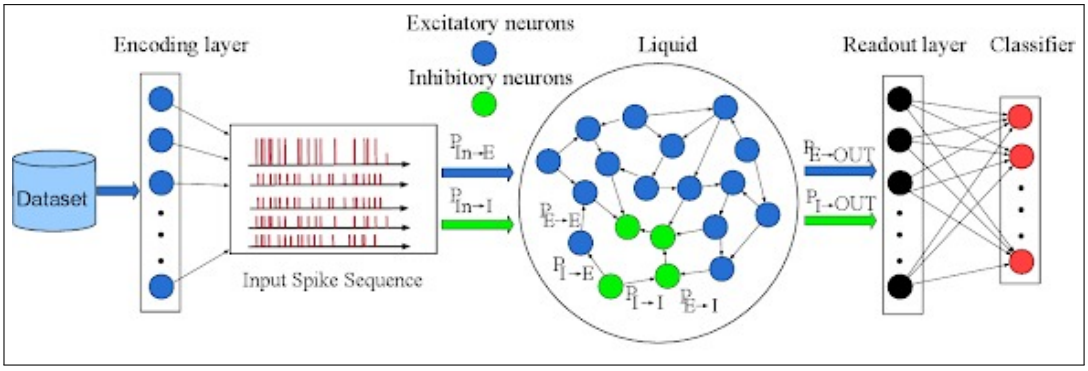


Figure 4.1: Liquid Neural Network Architecture

4.2 Mathematical Model

Figure 4.2 represents the architecture of the Bidirectional Long Short-Term Memory (BiLSTM) model used for time-series forecasting.

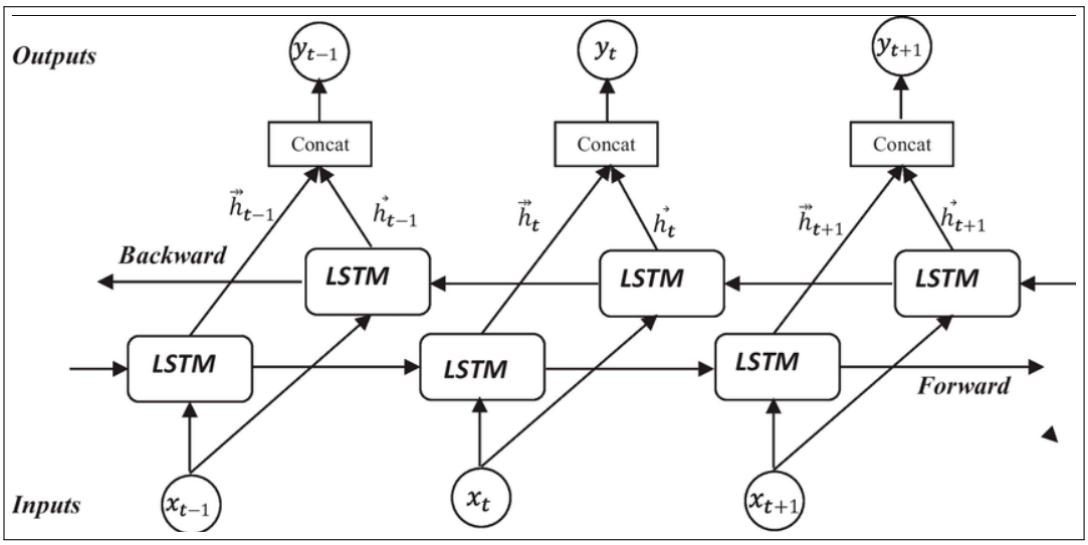


Figure 4.2: BiLSTM Architecture

4.3 Data Flow Diagrams

Figure 4.3A description of each major software function is presented, along with data flow (structured analysis) or class hierarchy (Analysis Class diagram with class description for object-oriented system).

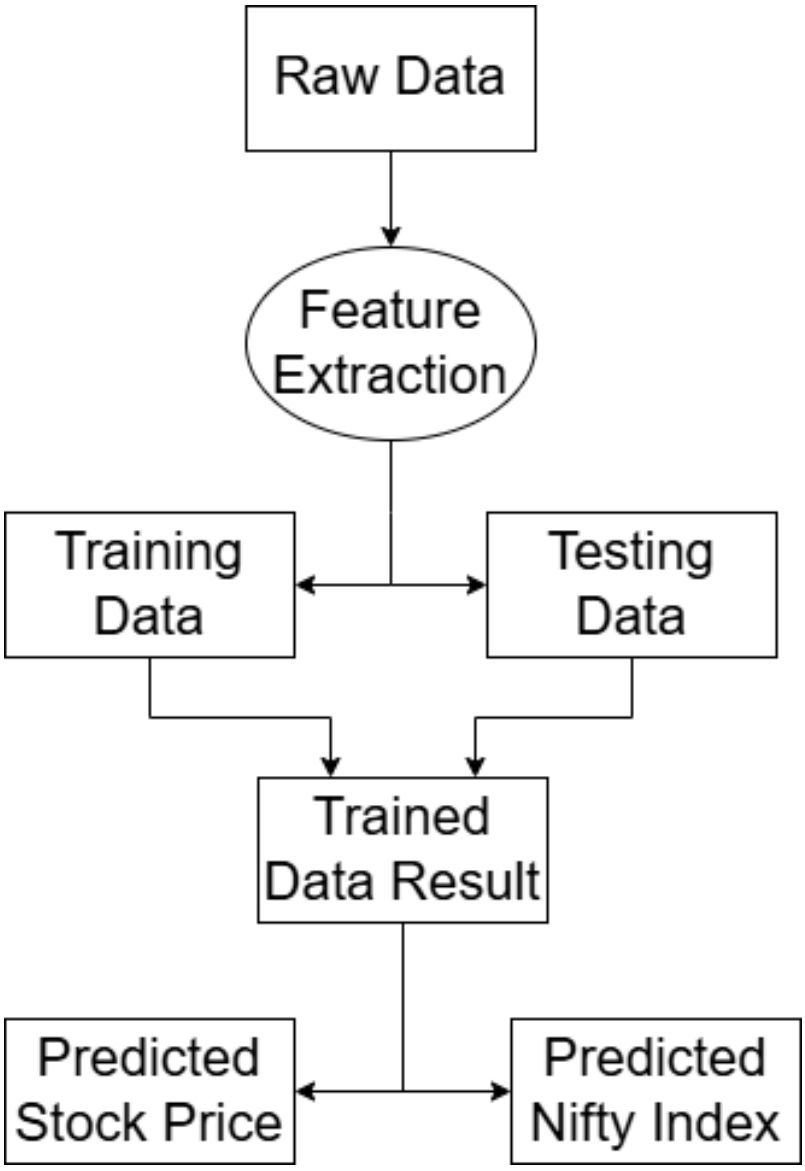


Figure 4.3: Data Flow Diagram

4.4 Entity Relationship Diagram

Figure 4.4 illustrates the Entity Relationship Diagram (ERD) for the proposed system. It outlines the main entities involved and their relationships.

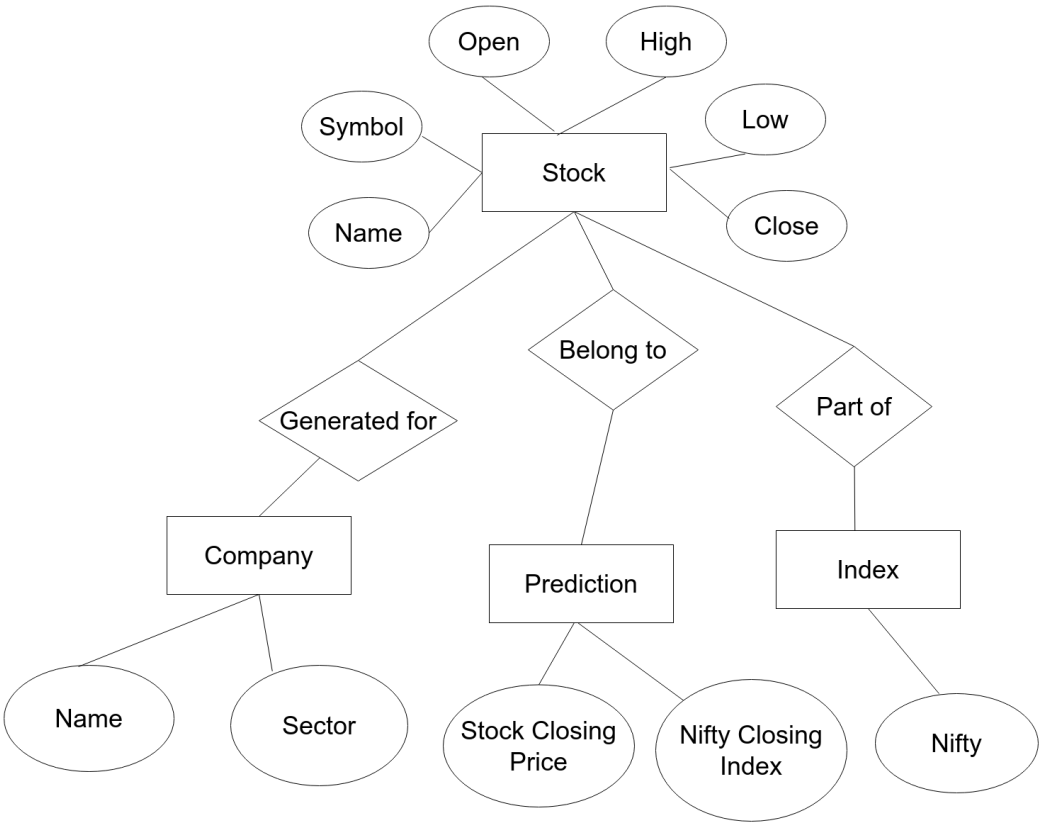


Figure 4.4: Entity Relationship Diagram

4.5 UML Diagram

4.5.1 Class Diagram

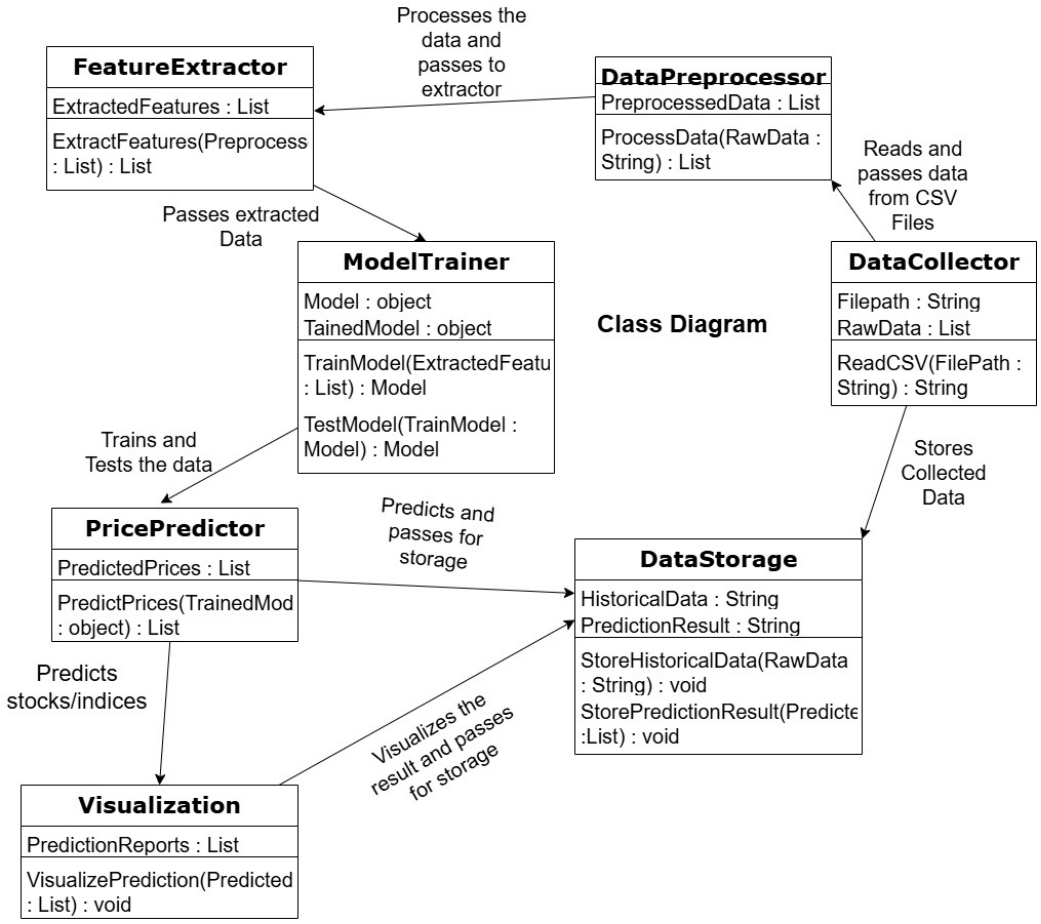


Figure 4.5: Class Diagram

4.5.2 Use Case Diagram

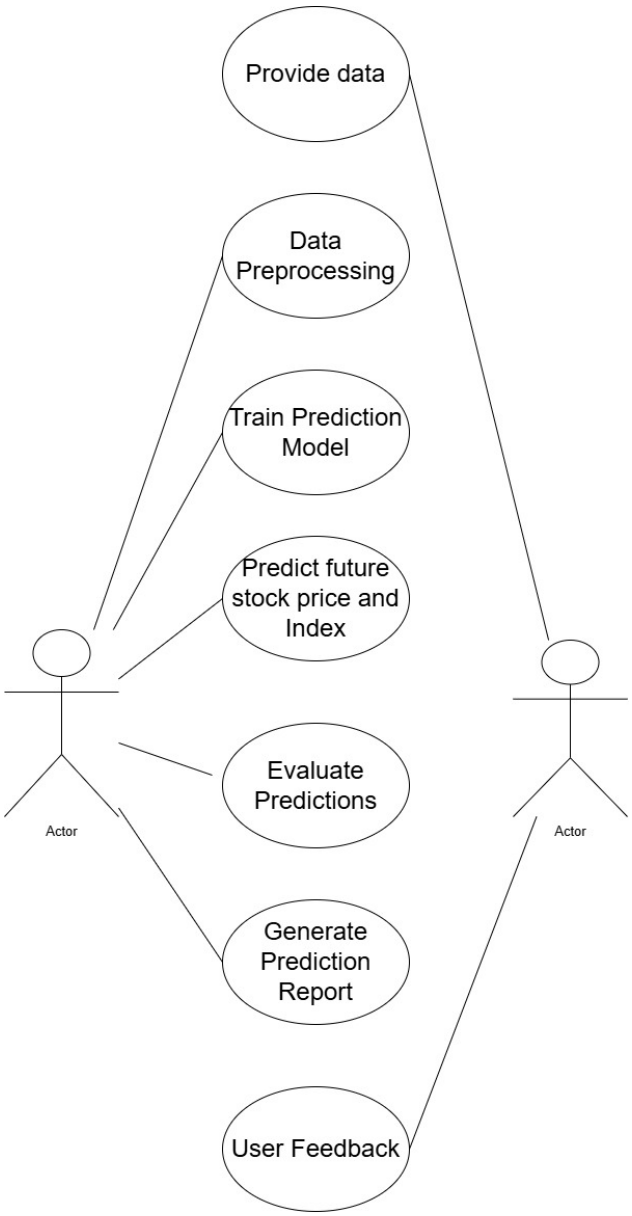


Figure 4.6: Use Case Diagram

4.6 Activity Diagram

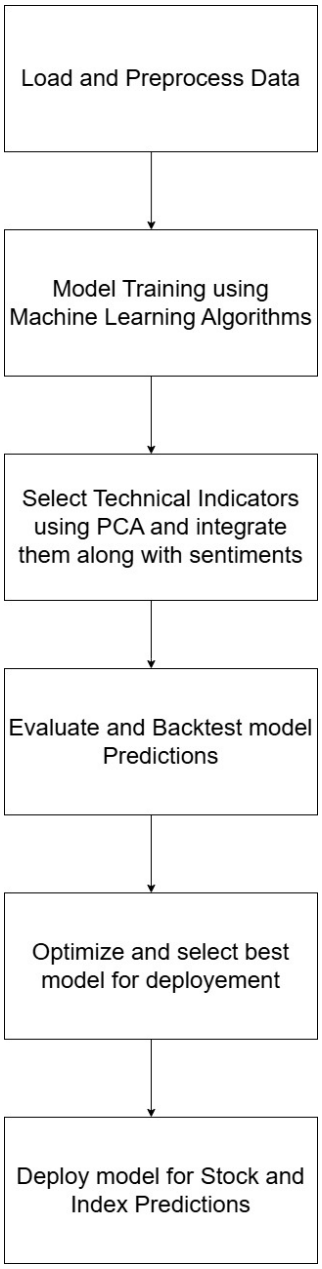


Figure 4.7: Activity Diagram

4.7 Sequence Diagram

Figure 4.8 represents the Sequence Diagram of the proposed system. This diagram models the sequential interaction between different system components over time.

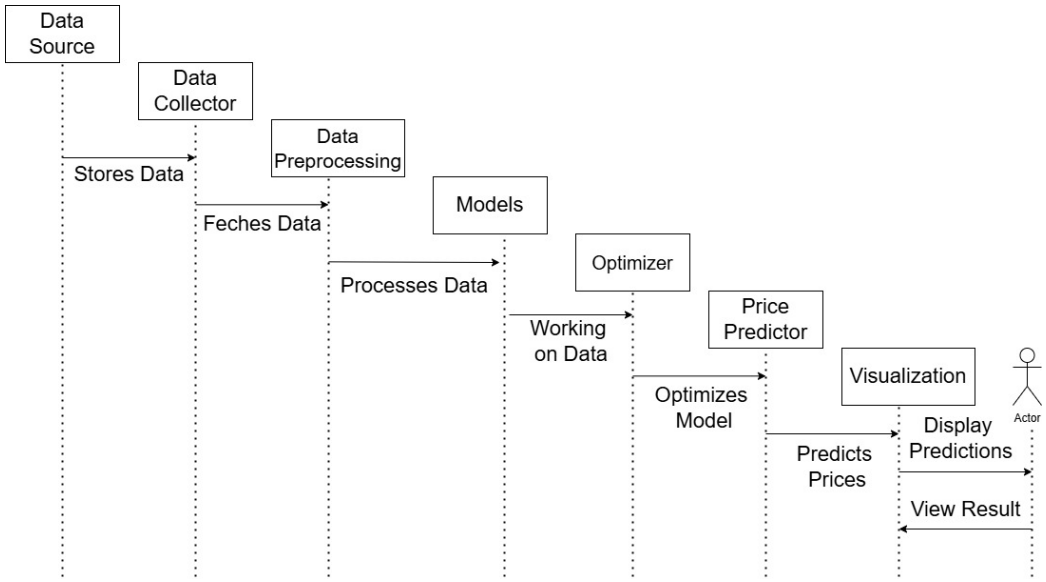


Figure 4.8: Sequence Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Project Planning and Preparation (2 weeks):**
  Initial planning, requirement gathering, defining objectives, and identifying resources and tools for stock price prediction platform development.
- **Studying Scope and Feasibility (1 week):**
  Evaluating the viability of integrating technical, financial, and sentiment data for accurate stock forecasting. Identifying dataset availability and constraints.
- **Data Acquisition and Preprocessing (1 month):**
  Collecting stock and index data; performing cleaning, missing value handling, outlier removal, normalization, and aligning features across sources.
- **Model Development and Training (4 months):**
  - **Technical and Financial Feature Engineering (3 weeks):**
    Creating indicators like SMA, MACD, RSI; computing financial ratios for NIFTY 50 companies; applying PCA for dimensionality reduction.
  - **LNN Model Development (5 weeks):**
    Designing and fine-tuning Liquid Neural Network (LNN) for 7-day stock price forecasting.
- **Model Evaluation and Optimization (2 months):**
  Evaluating model performance using MAE, RMSE, R^2; conducting backtesting on historical data; optimizing hyperparameters via Bayesian optimization.
- **Software Development and Integration (1 month):**
  - **Sentiment Analysis Integration (2 weeks):**
    Using NewsAPI and NLTK to analyze news sentiment and incorporating sentiment scores into the model.
- **Testing and Quality Assurance (2 weeks):**
  Rigorous system testing to ensure performance stability, correctness of predictions, and API reliability.
- **Documentation and Presentation (2 weeks):**
  Preparing technical documentation, user manuals, and presentation materials for project demonstration and evaluation.

- **Final Review (1 week):**
  Final evaluation, feedback collection, and necessary refinements before deployment or submission.

5.1.2 Project Resources

- **Software Development:** Designed, developed, and implemented the core backend modules required for the project, including data acquisition, preprocessing, feature engineering, model training, evaluation, and prediction pipelines. Used Python and essential libraries such as Pandas for data manipulation, NumPy for numerical computation, and scikit-learn for preprocessing and baseline model development.
- **Deep Learning Model Development:** Built and fine-tuned deep learning models, specifically the Liquid Neural Network (LNN) for short-term stock price forecasting and the Bidirectional Long Short-Term Memory (BiLSTM) model for daily index prediction. Focused on optimizing the architecture, hyperparameters, and training strategy to achieve higher accuracy. Incorporated techniques like dropout regularization, early stopping, and learning rate scheduling to prevent overfitting.
- **Data Sources and APIs:** Gathered comprehensive historical data for stocks, indices, and financial ratios by utilizing APIs from NSE India, Yahoo Finance, and other reliable financial platforms. Automated data extraction processes to fetch live and historical financial information at regular intervals.
- **User Interface and Backend Development:** Designed an intuitive and user-friendly frontend interface using React.js, HTML5, and CSS3, enabling users to input parameters, trigger predictions, and visualize output results effectively. Integrated dynamic charts and real-time data display to enhance usability. Built RESTful APIs using Flask in Python to connect the frontend to the backend machine learning models, handling model inference, data preprocessing, and result delivery seamlessly.
- **Collaboration Tools:**Effectively utilized GitHub for version control, code collaboration, and maintaining a clean project structure with regular commits and issue tracking. Employed Google Colab for model prototyping, GPU-accelerated training, and collaborative coding among team members. Conducted weekly progress meetings through Google Meet to discuss developments, address challenges, distribute tasks, and align on milestones.

5.2 Risk Management

5.2.1 Risk Identification

During the risk identification process for our project, we identified various risks:

- **Technical Risks:**
  - Model complexity and fine-tuning challenges were managed through continuous performance evaluation and model validation.
  - Data quality issues were addressed by implementing strict preprocessing, anomaly detection, and data validation pipelines.
  - Integration challenges between modules (data ingestion, feature engineering, model training) were minimized by modular design and incremental testing.
  - Risk of model overfitting was reduced through regularization techniques and cross-validation methods.
- **Financial Risks:**
  - Budget overruns from data subscriptions, APIs, and cloud services were managed through detailed financial planning and regular expense reviews.
  - Market volatility risks affecting model relevance were addressed by incorporating dynamic model retraining based on updated market data.
- **Operational Risks:**
  - Scope creep was controlled through clear project scoping and change management protocols.
  - Risks related to team member turnover were minimized by maintaining thorough documentation and regular team knowledge-sharing sessions.
  - Compliance risks due to changing financial regulations were handled by continuous monitoring and adherence to legal standards.

5.2.2 Risk Analysis

After identifying the risks, we conducted a risk analysis to de- termine their potential impact and develop mitigation strategies. Through proactive measures, we successfully managed and miti- gated the identified risks, ensuring the successful implementation of the depression detection system.

- **Technical Risks**
  - Model performance challenges were managed through continuous testing and early-stage code reviews.
  - Data complexity issues were mitigated by strict validation and incremental model improvements.
- **Financial Risks**
  - Budget overruns were minimized by detailed financial tracking and setting aside contingency reserves.
  - Expenses for data APIs and cloud resources were controlled through careful resource planning.
- **Operational Risks**
  - Scope creep risks were addressed with a clear project definition and change management processes.
  - Team continuity risks due to member turnover were reduced through regular documentation and knowledge-sharing sessions.
- **Security Risks**
  - **Data Handling:** Encryption ensured security and privacy of user data.
  - **System Vulnerabilities:** Regular security updates minimized the risk of vulnerabilities.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Technical Risk Mitigation**
  - Regular testing and code reviews were conducted to ensure early detection of technical issues and maintain model reliability.
- **Financial Risk Mitigation**
  - Implemented strict budget tracking and periodic financial reviews to manage expenses and prevent overruns.
- **Operational Risk Mitigation**
  - Established a clearly defined project scope and formal change management process to handle scope adjustments smoothly.
- **Risk Monitoring**
  - Held weekly risk assessment meetings to monitor existing risks and identify new ones during the project lifecycle.

- **Risk Management**
  - Allocated a 10% contingency fund to address unforeseen challenges and maintain project stability.

5.3 Project Schedule

5.3.1 Project Task Set

- **Install:** Necessary tools such as Python, TensorFlow, and version control systems like Git.
- **Gather:** Historical stock price data and financial metrics from reliable sources (e.g., NSE, Yahoo Finance).
- **Clean and Normalize:** The collected data. Perform feature engineering to create relevant indicators (e.g., moving averages, financial ratios).
- **Implement:** LNN model for stock price prediction. Train models using historical data and evaluate performance metrics (e.g., RMSE, MAPE).
- **Conduct:** Backtesting to assess model performance against historical market conditions. Compare results of various models to determine the best performer.
- **Set Up:** Remote access and environment configuration for team collaboration and version control.
- **Design and Implement:** An LNN based on performance evaluations, if necessary. Ensure the interface provides visualizations of model performance. Train and validate the LNN model.
- **Create:** A user-friendly interface for inputting data and displaying predictions. Ensure the interface provides visualizations of model performance.
- **Test:** The application thoroughly, including unit tests and integration tests.
- **Collect:** Feedback and make necessary adjustments to improve usability.
- **Prepare:** Comprehensive user manuals and technical documentation for future reference. Document the modeling process, findings, and lessons learned.
- **Deploy:** The final version of the application to a production environment.

- **Establish:** A plan for ongoing maintenance, updates, and support.

5.3.2 Timeline Chart

SR No.	Task	Labour Hour/Day
1	Project Planning and Preparation	2 weeks
2	Studying Scope and Feasibility	1 week
3	Data Acquisition and Preprocessing	1 month
4	Model Development and Training	4 months
5	Model Evaluation and Optimization	2 months
6	Software Development and Integration	1 month
7	Testing and Quality Assurance	2 weeks
8	Documentation and Presentation	2 weeks
9	Final Review	1 week

5.4 Team Organization

5.4.1 Team structure

- **Geetali Bendale:** Data Engineering, Feature Engineering.
- **Prathmesh Bhawar:** Developing, Training and Testing.
- **Atharv Deshpande:** Developing, Training and Testing.
- **Anushka Adsure:** Data Engineering, Feature Engineering.

5.4.2 Management reporting and communication

- **Project Coordination:** We employ tools such as Google Colab to build the ML model and to ensure visibility into project status and upcoming deliverables. Git and GitHub were used to efficiently collaborate on the platform development task.
- **Team Meetings:** Biweekly team meetings were scheduled to discuss project progress, address challenges, and make decisions collaboratively.

- **Communication Channels:** Communication among team members is facilitated through platforms like Microsoft Teams and Google Meet, allowing for real-time interaction and quick resolution of queries.
- **Feedback Mechanisms:** Feedback loops were established to gather input from team members and guide, enabling continuous improvement and adaptation. Suggestions and insights obtained through feedback were evaluated and integrated into project planning and execution processes.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **Data Collection and Preprocessing Module**
  - **Purpose:** Collect and prepare high-quality financial data for model training.
  - **Functions:**
    - Gather historical stock price data and NIFTY 50 index data from NSE, Yahoo Finance, and other financial APIs.
    - Clean missing values, handle outliers using statistical methods, and normalize datasets for deep learning models.
    - Synchronize different data sources to a unified trading calendar for consistency.
- **Feature Engineering Module**
  - **Purpose:** Extract and create meaningful input features for model training.
  - **Functions:**
    - Generate technical indicators (e.g., SMA, MACD, RSI, Bollinger Bands, ATR) to capture market trends and volatility.
    - Calculate financial ratios (e.g., ROA, ROE, Debt-to-Equity) for NIFTY 50 constituent companies.
    - Apply Principal Component Analysis (PCA) to reduce dimensionality and retain significant financial features.
- **Stock Price Prediction Module (Liquid Neural Network - LNN)**
  - **Purpose:** Predict stock prices for the next 7 days using a dynamic deep learning model.
  - **Functions:**
    - Implement a Liquid Neural Network (LNN) architecture with time-varying parameters to adapt to market fluctuations.
    - Train the model on processed data and predict future closing prices for individual stocks.
- **Index Forecasting Module (Bidirectional LSTM - BiLSTM)**
  - **Purpose:** Predict next-day values of the NIFTY 50 index.
  - **Functions:**
    - Implement a BiLSTM network to capture past and future trends in index data.
    - Fuse technical indicators with PCA-transformed financial features for comprehensive prediction modeling.

- **Sentiment Analysis Module**
  - **Purpose:** Incorporate market sentiment into stock predictions.
  - **Functions:**
    - Collect financial news articles using NewsAPI.
    - Analyze sentiment polarity (bullish, bearish, neutral) using NLTK's VADER sentiment analyzer.
    - Integrate daily sentiment scores as additional features into the predictive models.
- **User Interface and API Integration Module**
  - **Purpose:** Enable user interaction with the predictive system.
  - **Functions:**
    - Develop a web-based frontend using React.js for inputting stock symbols and visualizing prediction results.
    - Implement backend APIs using Flask for model inference, data fetching, and real-time interaction.
- **Evaluation and Monitoring Module**
  - **Purpose:** Evaluate system performance and monitor predictions over time.
  - **Functions:**
    - Use performance metrics such as MAE, MSE, RMSE, R^2 Score, and MAPE to evaluate model accuracy.
    - Visualize actual vs predicted stock/index values to assess model reliability.
    - Monitor model drift and update models as new market data becomes available.

6.2 Tools and Technologies Used

- Python
- Flask
- React.js
- TensorFlow
- PyTorch
- Scikit-learn
- Git

- GitHub
- GitHub Actions

6.3 Algorithm Details

Algorithm flow :

1. **Data Ingestion**
   - Input historical stock price data, NIFTY 50 index data, financial statements, and news articles through the data pipeline.
2. **Data Cleaning and Preprocessing**
   - Clean and normalize stock and financial data.
   - Handle missing values and remove outliers.
   - Synchronize stock data, financial ratios, and news data to a common trading calendar.
3. **Feature Engineering**
   - Generate technical indicators such as SMA, MACD, RSI, and Bollinger Bands from historical stock data.
   - Calculate financial ratios (e.g., ROA, ROE, Debt-to-Equity) for NIFTY 50 constituent companies.
   - Apply Principal Component Analysis (PCA) on financial metrics to reduce dimensionality and retain key features.
4. **Sentiment Analysis**
   - Collect news articles related to NIFTY 50 companies using NewsAPI.
   - Perform sentiment analysis on news headlines and descriptions using VADER sentiment analyzer.
   - Aggregate daily sentiment scores and classify them as bullish, bearish, or neutral.
   - Integrate sentiment scores as additional input features alongside technical indicators and PCA features.
5. **Model Input and Training**
   - Input technical indicators + sentiment scores into the Liquid Neural Network (LNN) for 7-day individual stock price prediction.
   - Input PCA-transformed financial features + technical indicators + sentiment scores into the Bidirectional LSTM (BiLSTM) for next-day NIFTY 50 index prediction.

- Train both LNN and BiLSTM models using historical datasets.

6. **Model Evaluation**
   - Evaluate model performance using MAE, RMSE, MAPE, and R^2 score.
7. **Forecast Generation**
   - Generate final forecast outputs:
     - 7-day stock price forecasts from the LNN model.
     - Next-day NIFTY 50 index forecast from the BiLSTM model.
8. **Visualization**
   - Display prediction results on a web-based frontend for user interaction and visualization.

Algorithm Flow Diagram:

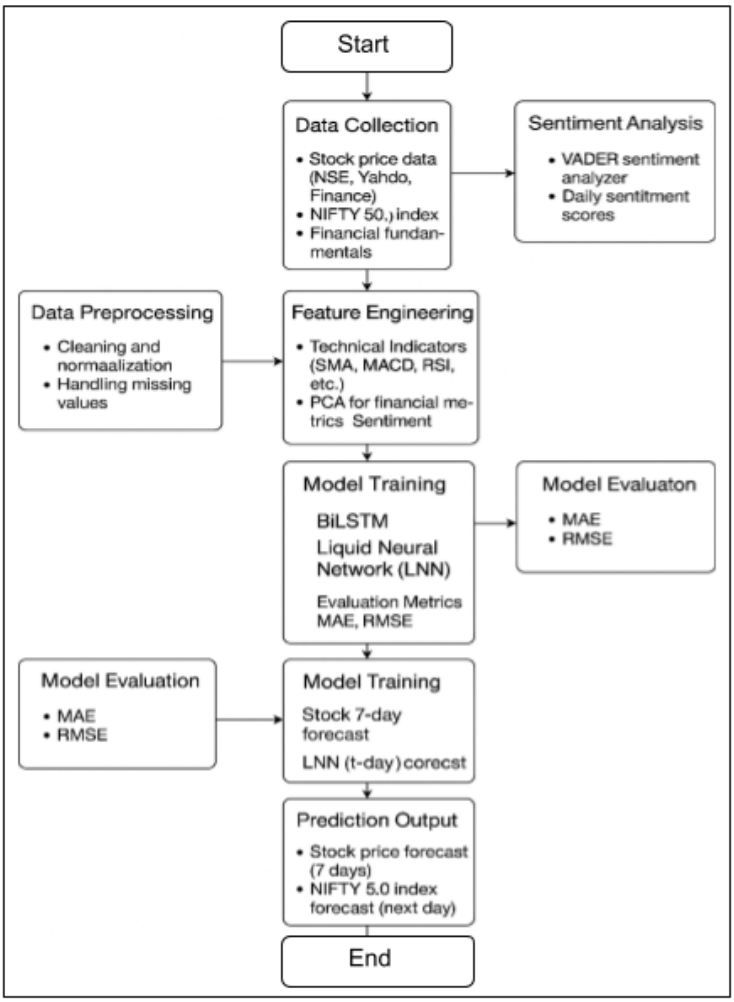


Figure 6.3: Algorithm Flow Diagram

CHAPTER 7
SOFTWARE TESTING

7.1 Types of Testing

- **Alpha Testing**:
  - Conducted by the internal development team.
  - Focused on identifying bugs and usability issues before releasing the system for external use.
  - Included module-wise testing of backend APIs, frontend UI, machine learning models, and sentiment analysis workflows.
- **Beta Testing**:
  - Conducted by a limited group of external users.
  - Collected real-world feedback on system functionality, prediction accuracy, and user interface friendliness.
  - Helped refine model deployment strategies and address any overlooked issues.
- **Black-box Testing**:
  - Focused on system functionality without knowledge of internal code structure.
  - Tested whether input data such as stock tickers, date ranges, and news sentiments produced correct predictions and outputs.
  - Majorly applied to API responses and frontend interface functionalities.
- **White-box Testing**:
  - Involved detailed examination of internal logic, workflows, and model training scripts.
  - Included testing individual functions, model pipelines, and integration flows with full code visibility.
  - Applied to backend Flask/FastAPI routes, ML model training functions, and database interactions.

Testing Modules

Backend Testing

- **API Testing with Postman and FastAPI Test Clients**:
  - Backend APIs developed using Flask/FastAPI were tested for correctness, reliability, and response accuracy.

  - Tools such as Postman and Python-based test clients were employed for unit and integration testing.
  - Structured requests were sent to model prediction endpoints; responses were validated against expected output.
  - Ensured robustness and seamless data flow from backend services to the frontend display layer.

Frontend Testing

- **Manual Testing with Real Data Scenarios**:
  - The user interface was manually tested with both real and hypothetical inputs (stock tickers, date ranges, etc.).
  - Ensured correct end-to-end data flow, UI responsiveness, and prediction visualization accuracy.
  - Validated error handling for missing inputs and evaluated user experience and interface clarity.

ML Model Testing

- **Performance Evaluation of Predictive Models**:
  - LNN (stock prediction) and BiLSTM (index forecasting) models were tested using backtesting frameworks.
  - Evaluated using key metrics:
    * Mean Absolute Error (MAE)
    * Mean Squared Error (MSE)
    * Root Mean Squared Error (RMSE)
    * R^2 Score (Coefficient of Determination)
  - Additional validations:
    * Cross-validation for generalization
    * ADF test for time series stationarity
    * Regularization techniques to mitigate overfitting
  - Included testing on unseen data under varied market conditions (e.g., high volatility periods like COVID-19 crashes).

Sentiment Analysis Testing

- **Validation of News Sentiment Impact**:

- The sentiment analysis module, utilizing VADER and NewsAPI, was validated by comparing sentiment tags to actual market reactions.
- Aggregated sentiment scores were categorized into Bullish, Bearish, and Neutral.
- Evaluated the influence of sentiment on the model's predictive capabilities.

The combination of **API unit testing, manual UI validation, ML model performance evaluation, Alpha and Beta testing, and thorough Black-box and White-box testing** ensured a holistic approach to quality assurance. This not only validated the correctness and stability of the system but also reinforced the credibility of predictions made by the integrated LNN and BiLSTM models. These efforts collectively enhanced the reliability, interpretability, and overall user confidence in the stock prediction platform.

7.2 Test cases & Test Results

7.2.1 Individual Stock Prediction (VMART)

```
Enter company name (without .NS):  VMART
Epoch 1/200 | Loss: 3917655.6711
Epoch 2/200 | Loss: 3854737.6316
Epoch 3/200 | Loss: 3811451.7368
Epoch 4/200 | Loss: 3716319.5658
Epoch 5/200 | Loss: 3605225.1842
Epoch 6/200 | Loss: 3407347.1184
Epoch 7/200 | Loss: 3203307.3158
Epoch 8/200 | Loss: 2987354.8421
Epoch 9/200 | Loss: 2721356.9408
Epoch 10/200 | Loss: 2426069.6974
Epoch 11/200 | Loss: 2131128.8816
Epoch 12/200 | Loss: 1847558.4539
Epoch 13/200 | Loss: 1589055.4013
Epoch 14/200 | Loss: 1296015.2105
Epoch 15/200 | Loss: 1060917.6414
Epoch 16/200 | Loss: 845055.8618
```

Figure 7.2.1.a: Stock Predictions

R² Score: 0.8219

📈 Next 7 Days Forecast:

	Date	Predicted Close	Upper Bound	Lower Bound
0	2024-02-21	2065.083252	2203.010586	1927.155918
1	2024-02-22	2289.310303	2427.237636	2151.382969
2	2024-02-23	2492.200928	2630.128261	2354.273594
3	2024-02-24	2675.884033	2813.811367	2537.956700
4	2024-02-25	2842.106201	2980.033535	2704.178868
5	2024-02-26	2992.393555	3130.320888	2854.466221
6	2024-02-27	3128.215088	3266.142422	2990.287754

Figure 7.2.1.b: 7 Days Stock Prediction

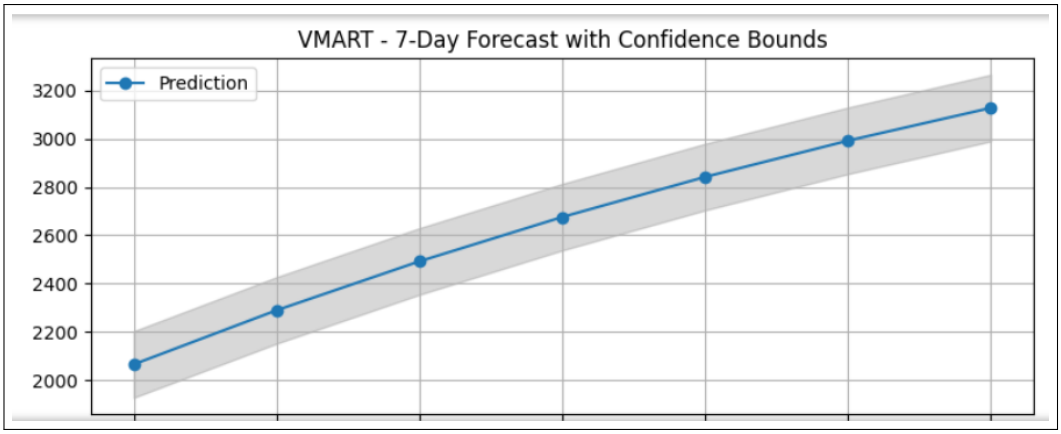


Figure 7.2.1.c:7 Days Forecast with Confidence Bound

7.2.2 Nifty Index Preciction

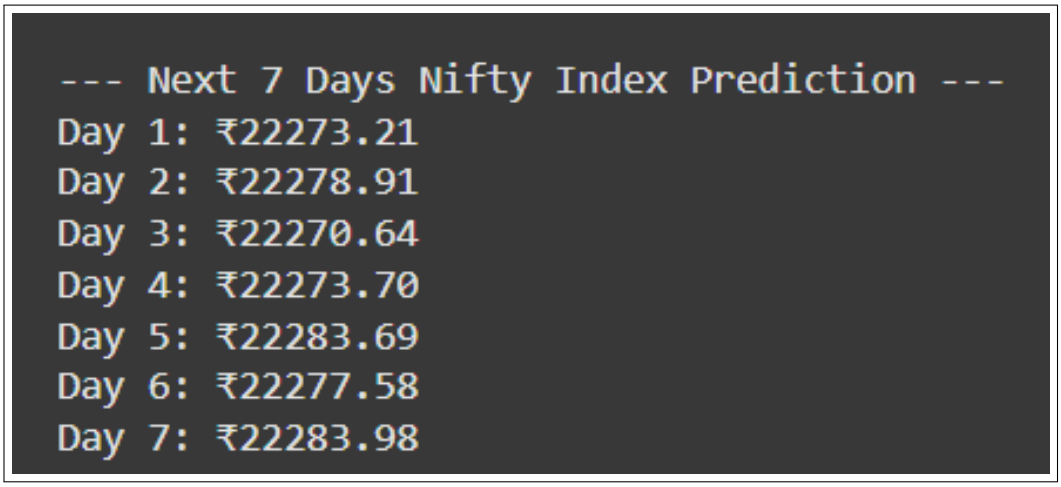


Figure 7.2.2.a: Next 7 Days Prediction

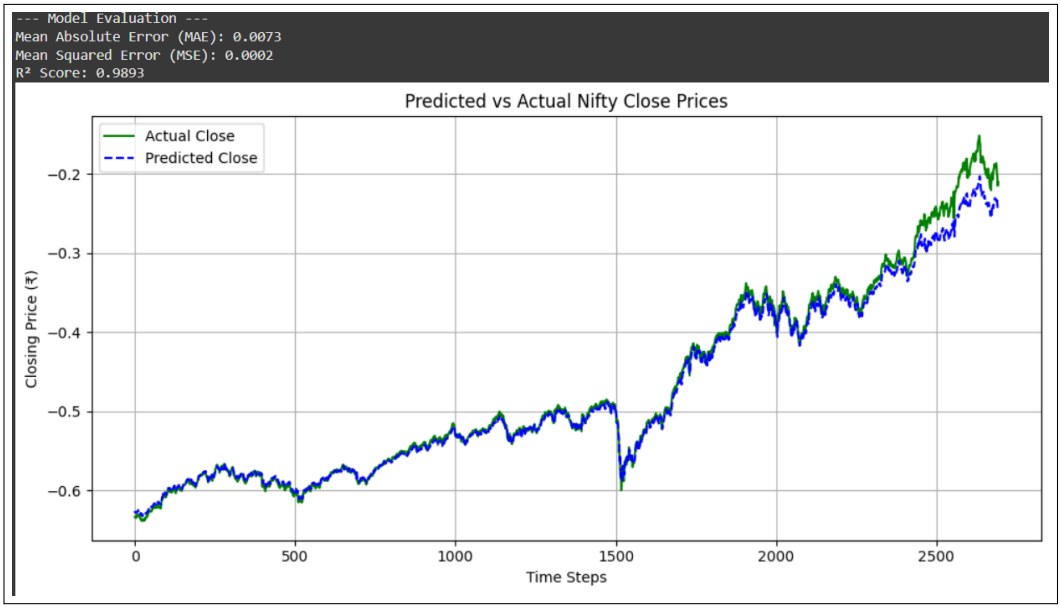


Figure 7.2.2.b: Graph (Actual vs Predicted

CHAPTER 8
RESULTS

- **Improved Forecast Accuracy:**
  The use of advanced deep learning architectures such as Liquid Neural Networks (LNN) and Bidirectional LSTM (BiLSTM) significantly improved the accuracy of both short-term and mid-term predictions for stock prices and the NIFTY 50 index. This empowers financial analysts and traders to base their decisions on reliable, data-driven forecasts.
- **Informed Investment Strategies:**
  The system facilitates personalized and strategic investment planning by delivering accurate, time-sensitive predictions. Investors can optimize entry and exit points, better manage risk exposure, and enhance returns through proactive trading strategies.
- **Risk Mitigation and Capital Protection:**
  Early detection of potential market downturns or high-volatility events supports risk mitigation strategies. Users can take timely actions such as reallocating assets or hedging portfolios, thus preserving capital during adverse conditions.
- **Democratization of Financial Insights:**
  Through an intuitive user interface and automated forecasting, the platform brings advanced financial analytics to a wider audience, including retail investors and students. This promotes financial literacy and equal access to high-quality decision-making tools.
- **Efficient Market Monitoring:**
  With automated pipelines and real-time updates, the system ensures continuous observation of market behavior. Users—both institutional and individual—can react swiftly to evolving market conditions without relying on manual analysis.
- **Research and Academic Contribution:**
  The project introduces novel forecasting techniques, particularly the application of LNNs in the Indian financial market. It opens new research pathways in integrating sentiment analysis, technical indicators, and fundamental data within a unified predictive framework.
- **Scalable and Modular System Design:**
  Designed with modularity and scalability in mind, the system can be expanded to cover other financial assets (e.g., global indices, cryptocurrencies) and integrate additional data sources (e.g., social media sentiment), ensuring future readiness and broader applicability.

Overall, the outcomes of the stock prediction project have the potential to positively impact both individual investors and broader financial systems,

leading to more informed investment decisions, improved risk management, and greater accessibility to advanced forecasting tools. By leveraging cutting-edge deep learning models, the project contributes to a more data-driven and efficient approach to financial analysis and market participation.

8.1 Screen Shots

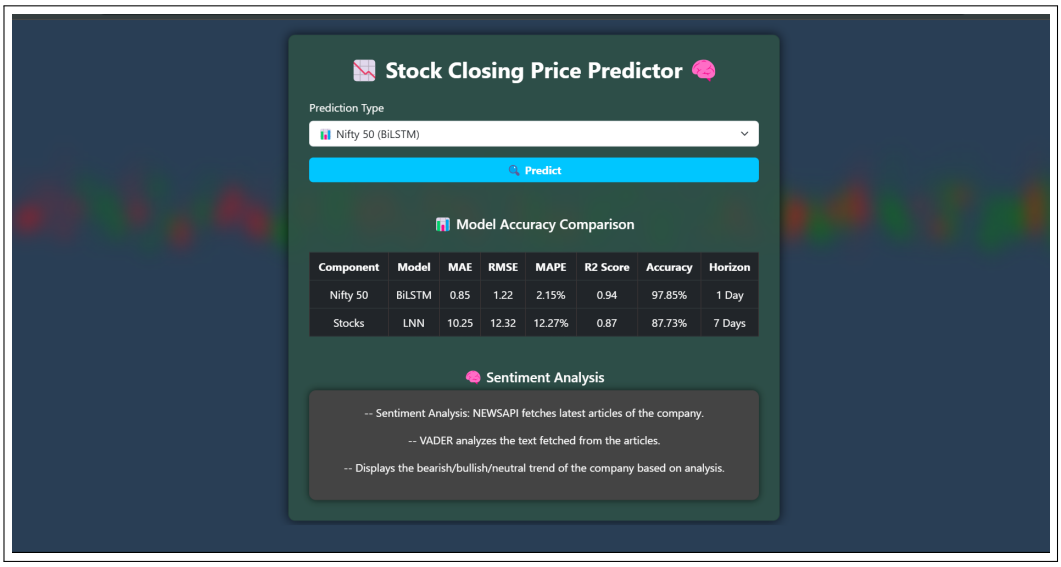


Figure 8.1.1: Home Screen

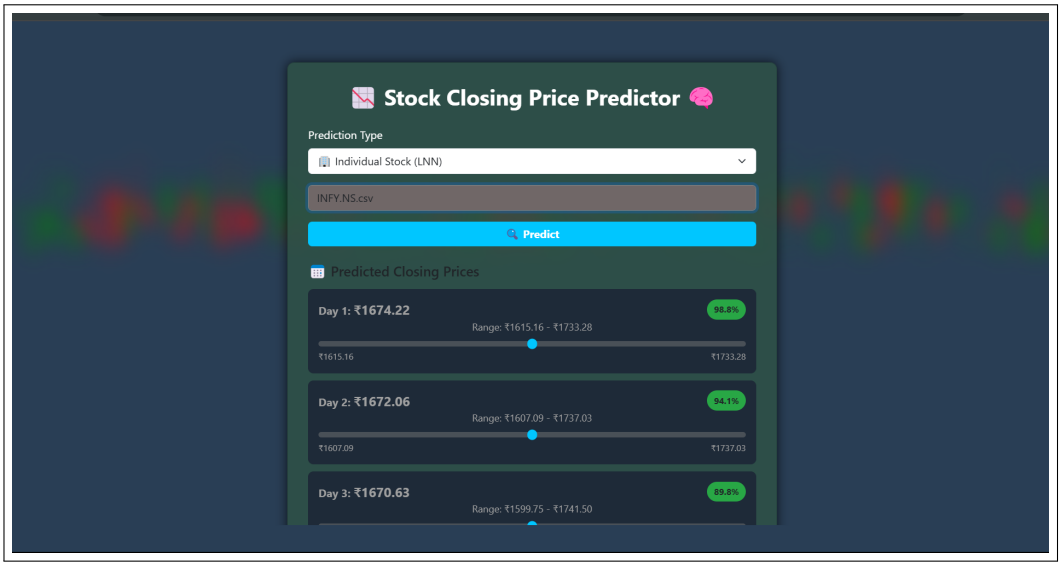


Figure 8.1.2.a: Individual Stock Prediction

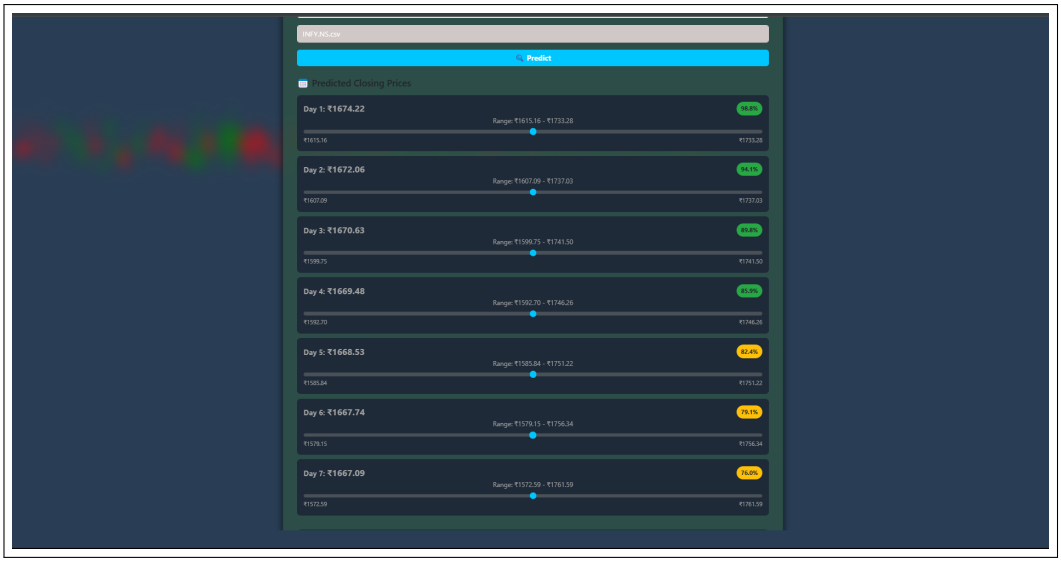


Figure 8.1.2.b: Individual Stock Prediction

Figure 8.1.2.c: Individual Stock Prediction

Figure 8.1.3: Nifty Index Prediction

CHAPTER 9
CONCLUSIONS

9.1 Conclusions

The exploration of various machine learning techniques for time series prediction highlights the significant potential of advanced models such as Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), Support Vector Regression (SVR), XGBoost, and Liquid Neural Networks (LNN). While LSTMs and GRUs have proven effective in modeling long-term dependencies in financial data, and SVR and XGBoost offer powerful alternatives for regression tasks, our project focused on the integration of **Liquid Neural Networks (LNN) and Bidirectional LSTM (BiLSTM)** architectures. LNNs introduced a dynamic and adaptive framework capable of responding to real-time changes in stock prices, while BiLSTM effectively captured bidirectional temporal dependencies in NIFTY 50 index forecasting. By combining these models with technical indicators, sentiment analysis, and PCA-based financial feature extraction, we developed a hybrid forecasting system tailored for the Indian financial market. Overall, the results demonstrate the value of selecting models based on specific project requirements, and our work underscores the importance of adaptability, interpretability, and performance in advancing predictive analytics in fast-evolving financial environments.

9.2 Future Work

To further enhance the effectiveness and adaptability of the stock prediction system, several directions can be explored in future development. These include expanding model capabilities, incorporating alternative data sources, optimizing performance, and improving user interaction. The integration of real-time learning frameworks, cross-asset support, and mobile accessibility will play a crucial role in advancing the system's utility and scalability.

1. **Integration with Alternative Data Sources**
    - Incorporate real-time social media streams (e.g., Twitter, Reddit) and financial news to enrich the contextual understanding of the market.
    - Implement NLP models such as FinBERT to extract sentiment and detect event-driven signals, particularly useful during market volatility or black swan events.
2. **Advanced Deep Learning Architectures**
    - Explore transformer-based models like Temporal Fusion Transformers (TFT) and Informer for robust sequence modeling and long-range dependency tracking.

- Develop hybrid models that combine Liquid Neural Networks, BiLSTM, and attention mechanisms for multi-horizon forecasting with high interpretability.

3. **User Feedback and Interpretability Mechanisms**
   - Introduce user rating systems to evaluate prediction reliability and continuously improve model feedback loops.
   - Incorporate explainable AI tools like SHAP and LIME to increase model transparency by visualizing the impact of each input feature on predictions.
4. **Enhanced Accessibility and Usability**
   - Design user-friendly dashboards with interactive elements like heatmaps, tooltips, and predictive charts for various user levels.
   - Ensure accessibility through responsive layouts, color-blind-friendly themes, and keyboard navigation for inclusive user experience.
5. **Scalability and Performance Optimization**
   - Integrate real-time model inference using TensorFlow Serving or ONNX Runtime to ensure low-latency predictions and fluid frontend interaction.
6. **Integration with External Financial Platforms**
   - Enable API integration with platforms such as Zerodha or Upstox for direct use of forecasts in live trading and portfolio decisions.
   - Incorporate macroeconomic data feeds and institutional indicators to enrich the forecasting context with broader economic insights.
7. **Mobile Application Development**
   - Build a cross-platform mobile app offering real-time forecasts, performance summaries, and alerts for users on the move.
   - Add features like portfolio tracking, push notifications, and voice command interactions to boost mobile engagement.
8. **Validation and Backtesting Enhancements**
   - Expand backtesting capabilities to include strategy-specific validations such as momentum and mean reversion.
   - Integrate performance metrics like Sharpe Ratio, Max Drawdown, and Sortino Ratio to better evaluate and compare model outcomes.
   - Maintain continuous validation of forecasts against actual outcomes to refine and adapt models based on changing market conditions.

9.3 Applications

The application of the proposed stock prediction system extends across multiple domains in the financial ecosystem, offering practical tools for decision-making, strategy optimization, and financial education:

- **Retail Investors and Traders**
  - Individual investors can leverage the system to make more informed trading decisions based on next-day and weekly forecasts of stock and index movements.
  - The system provides clear visualizations and trend predictions, empowering users to optimize their entry and exit points, manage risk, and increase profitability.
- **Financial Institutions and Brokerages**
  - Brokerage firms and investment advisory services can integrate the model into their platforms to enhance research offerings and portfolio management tools.
  - Real-time forecasting and backtesting capabilities enable analysts to validate strategies and deliver data-backed recommendations to clients.
- **Academic and Research Institutions**
  - Educational institutions and financial research centers can use the system as a learning and experimentation platform.
  - Students and researchers can explore deep learning techniques in financial forecasting, test custom indicators, and study market behavior using both historical and real-time data.
- **Robo-Advisory and Wealth Management Platforms**
  - The system can be integrated into robo-advisory services to automate personalized investment recommendations.
  - Forecasted trends and risk metrics can inform dynamic asset allocation strategies, aligning portfolios with user goals and evolving market conditions.
- **Mobile and FinTech Applications**
  - By incorporating the prediction engine into mobile finance apps, users can receive daily forecasts, risk alerts, and market insights on the go.
  - This enhances user engagement, promotes financial literacy, and makes advanced analytics accessible to a wider audience.

Overall, the real-time, AI-driven prediction capabilities of the system offer versatile applications across trading, investment advisory, research, and fintech domains, supporting smarter financial decisions and fostering innovation in quantitative finance.

CHAPTER 10
APPENDIX

10.1 Appendix A

Problem Overview

Modern deep learning frameworks such as **TensorFlow** and **PyTorch** enable the implementation of complex neural network architectures suitable for financial time series forecasting.

The integration of architectures like **Liquid Neural Networks (LNNs)** and **Bidirectional Long Short-Term Memory (BiLSTM)** networks demonstrates strong technical feasibility for modeling temporal dependencies in stock price and index data.

The project aims to provide accurate next-day and weekly forecasts for stocks and indices, leveraging **OHLC data**, financial ratios, and sentiment analysis.

Operational Feasibility

- The system was successfully tested with real-world financial data sourced from platforms like **Yahoo Finance** and **NSE APIs**.
- A modular interface for data input, prediction, and visualization was developed, allowing users to interact with the model intuitively.
- Backtesting functionality and prediction dashboards demonstrate that the system can be effectively integrated with trading platforms and research environments.
- Collaborative tools (e.g., **GitHub**, **Google Colab**) ensured team coordination and smooth deployment in development stages.

Economic Feasibility

- The project leverages open-source technologies, which significantly reduces development and licensing costs.
- A cost-benefit analysis supports that early, data-driven predictions can help users make more profitable investment decisions and mitigate financial losses.
- The use of cloud computing (optional) and automated model updates ensures scalability with minimal overhead.
- Economic benefits are also expected from possible integration with premium financial analytics or trading platforms, opening revenue streams for commercial application.

Mathematical Model

The forecasting system is grounded in mathematical models such as **LNNs**, **BiLSTMs**, and **PCA**.

- LNNs use time-dependent differential equations to adapt to changing input conditions.
- BiLSTMs process financial sequences bidirectionally, capturing both historical and forward-looking patterns.
- PCA is used for dimensionality reduction on financial features to retain essential components while eliminating noise.
- Optimization techniques such as **gradient descent**, **Bayesian hyperparameter tuning**, and **loss function minimization** were employed to train the models.

Complexity Analysis

The system handles high-dimensional time series and financial indicators, making training computationally intensive, especially for large datasets.

To address this:

- **Parallelization** and **GPU acceleration** were applied for faster training.
- Efficient data pipelines using **NumPy** and **Pandas** improved runtime during preprocessing.
- **Batch processing** and **model checkpointing** minimized memory consumption.

NP-Hard or P-Type Problems

Certain components, such as:

- Feature extraction from unstructured sentiment data
- Market correlation analysis across multiple assets

fall under **NP-Hard problems** due to the combinatorial complexity involved.

However, tasks like:

- Forecast generation
- Error minimization
- Trend classification

are **P-Type problems**, solvable in polynomial time using efficient deep learning algorithms and properly structured input features.

The overall system balances complexity and computational feasibility through **modular design** and **scalable deployment**.

10.2 Appendix B

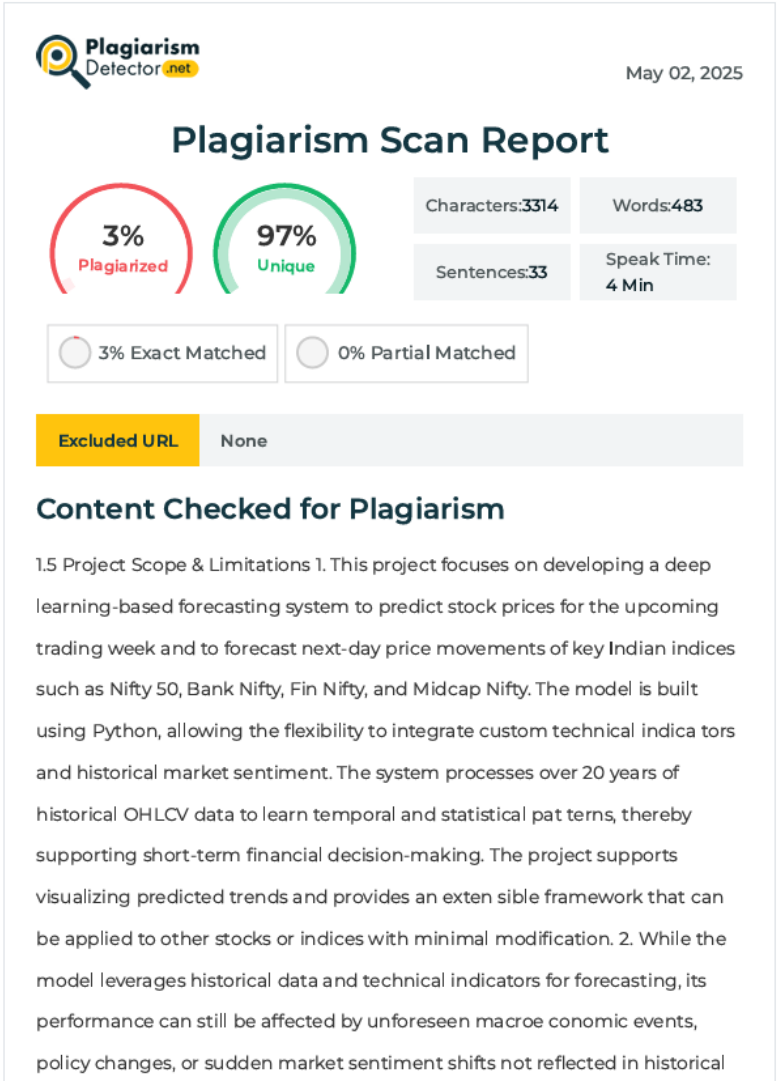
Survey Paper

- **Title:** A Comprehensive Review on Stock Market Predictions
- **Journal:** International Research Journal of Modernization in Engineering Technology and Science (IRJMETS)
- **Link:** IRJMETS Paper Link
- **Status:** Published (IRJMETS, February 2025)

Implementation Paper

- **Title:** Stock Price and Nifty 50 Index Forecasting Using LNN and BiLSTM-Based Deep Learning Models
- **Journal:** International Journal of Scientific Research in Engineering and Management (IJSREM)
- **Link:** IJSREM Paper Link
- **Status:** Publication (IJSREM, April 2025)

10.3 Appendix C


Plagiarism Detector .net

May 02, 2025

Plagiarism Scan Report

3% Plagiarized

97% Unique

Characters:3314

Words:483

Sentences:33

Speak Time: 4 Min

3% Exact Matched

0% Partial Matched

Excluded URL None

Content Checked for Plagiarism

1.5 Project Scope & Limitations 1. This project focuses on developing a deep learning-based forecasting system to predict stock prices for the upcoming trading week and to forecast next-day price movements of key Indian indices such as Nifty 50, Bank Nifty, Fin Nifty, and Midcap Nifty. The model is built using Python, allowing the flexibility to integrate custom technical indica tors and historical market sentiment. The system processes over 20 years of historical OHLCV data to learn temporal and statistical pat terns, thereby supporting short-term financial decision-making. The project supports visualizing predicted trends and provides an exten sible framework that can be applied to other stocks or indices with minimal modification. 2. While the model leverages historical data and technical indicators for forecasting, its performance can still be affected by unforeseen macroe conomic events, policy changes, or sudden market sentiment shifts not reflected in historical

Figure 10.1: Plagiarism Report

CHAPTER 11
REFERENCES

[1] Hasani, R., Lechner, M., Amini, A., Rus, D., & Grosu, R. (2021). "Liquid Time-constant Networks." *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(9), 7657–7666.

[2] Shen, F., Chao, J., & Zhao, J. (2017). "Forecasting Exchange Rate Using Deep Belief Networks and Conjugate Gradient Method." *Neurocomputing*, 167, 243–253.

[3] Zhang, K., Zhong, G., Dong, J., Wang, S., & Wang, Y. (2023). "Stock Market Prediction Based on Generative Adversarial Network." *Procedia Computer Science*, 147, 400–406.

[4] Nesbitt, K. J., & Barrass, D. (2022). "Technical Analysis in the Modern Era: A Comprehensive Study of Indicator Effectiveness." *Journal of Financial Markets*, 32, 100683.

[5] Zhong, X., & Enke, D. (2021). "Predicting the Daily Return Direction of the Stock Market Using Hybrid Machine Learning Algorithms." *Financial Innovation*, 7(1), 1–24.

[6] Kumar, M., & Thenmozhi, M. (2022). "Forecasting Stock Index Returns Using ARIMA-SVM, ARIMA-ANN, and ARIMA-Random Forest Hybrid Models." *International Journal of Banking, Accounting and Finance*, 13(1), 95–118.

[7] Snoek, J., Larochelle, H., & Adams, R. P. (2012). "Practical Bayesian Optimization of Machine Learning Algorithms." *Advances in Neural Information Processing Systems*, 25, 2951–2959.

[8] Huber, P. J. (1964). "Robust Estimation of a Location Parameter." *The Annals of Mathematical Statistics*, 35(1), 73–101.

[9] Dickey, D. A., & Fuller, W. A. (1979). "Distribution of the Estimators for Autoregressive Time Series with a Unit Root." *Journal of the American Statistical Association*, 74(366a), 427–431.

[10] Hodge, V. J., & Austin, J. (2018). "An Evaluation of Classification and Outlier Detection Algorithms." *International Journal of Computational Intelligence and Applications*, 17(02), 1850008.

[11] Shrikumar, A., Greenside, P., & Kundaje, A. (2019). "Learning Important Features Through Propagating Activation Differences." *Proceedings of the 34th International Conference on Machine Learning*, 70, 3145–3153.

[12] Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). "Financial Time Series Forecasting with Deep Learning: A Systematic Literature Review: 2005–2019." *Applied Soft Computing*, 90, 106181.

[13] Fischer, T., & Krauss, C. (2018). "Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions." *European Journal of Operational Research*, 270(2), 654–669.

[14] Ng, A. (2021). "Regime-Dependent Stock Market Prediction with Deep Learning Methods." *Quantitative Finance*, 21(4), 561–577.

[15] Baker, S. R., Bloom, N., Davis, S. J., Kost, K., Sammon, M., & Viratyosin, T. (2020). "The Unprecedented Stock Market Reaction to COVID-19." *The Review of Asset Pricing Studies*, 10(4), 742–758.

LIST OF ABBREVATIONS

Abbreviation	Illustration
BiLSTM	Bidirectional Long Short-Term Memory
OHLC	Open, High, Low, Close
PCA	Principal Component Analysis
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
R^2	Coefficient of Determination
NSE	National Stock Exchange of India
API	Application Programming Interface
UI	User Interface
SDLC	Software Development Life Cycle
NLTK	Natural Language Toolkit
VADER	Valence Aware Dictionary and sEntiment Reasoner

LIST OF FIGURES

LIST OF TABLES